

DevLoop — agentic software engineering workflow

A brief introduction to DevLoop — a proof-of-concept for agentic software engineering workflow with focus on sustainability, efficiency, and traceability.

Author: Ingo Dering, TEMI, 17.03.2026

⚙️ Small steps · ✅ Verified quality · 🔗 Full traceability

Use ← → arrow keys or buttons to navigate

What Is DevLoop?

DevLoop is an **agentic software engineering workflow prototype** that turns software delivery into a controlled, step-by-step process.

Instead of...

-  Writing all code at once and hoping it works
-  Manual handoffs between team members
-  Discovering problems late in the process

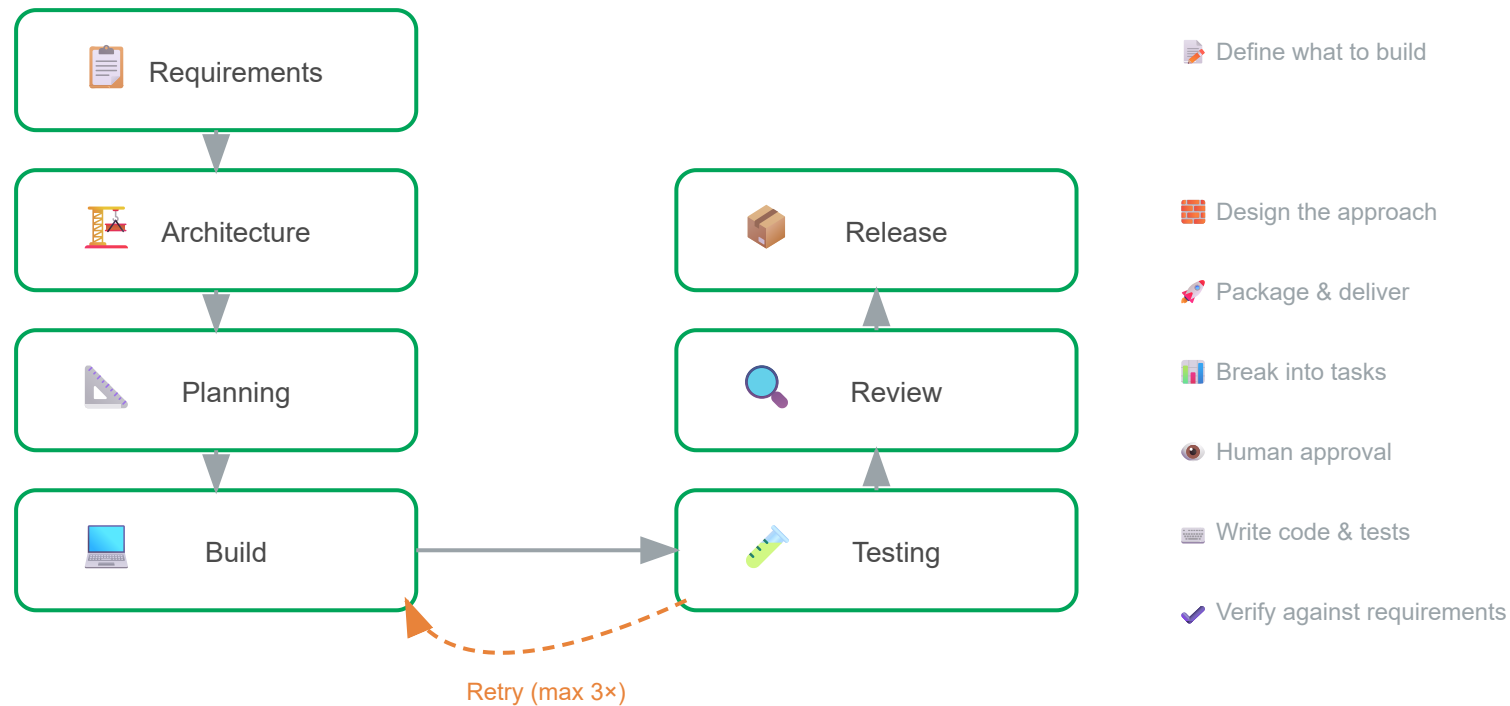
DevLoop provides...

-  Manageable pieces tested against requirements
-  Automated, document-driven handoffs
-  Quality checks at every transition

 **Think of it as a software assembly line** — each stage has a specialist, a defined handoff, and a quality check before the next stage can begin.

The Development Phases

Each feature moves through **7 fixed phases**. The workflow only advances when the current phase has been completed and checked.





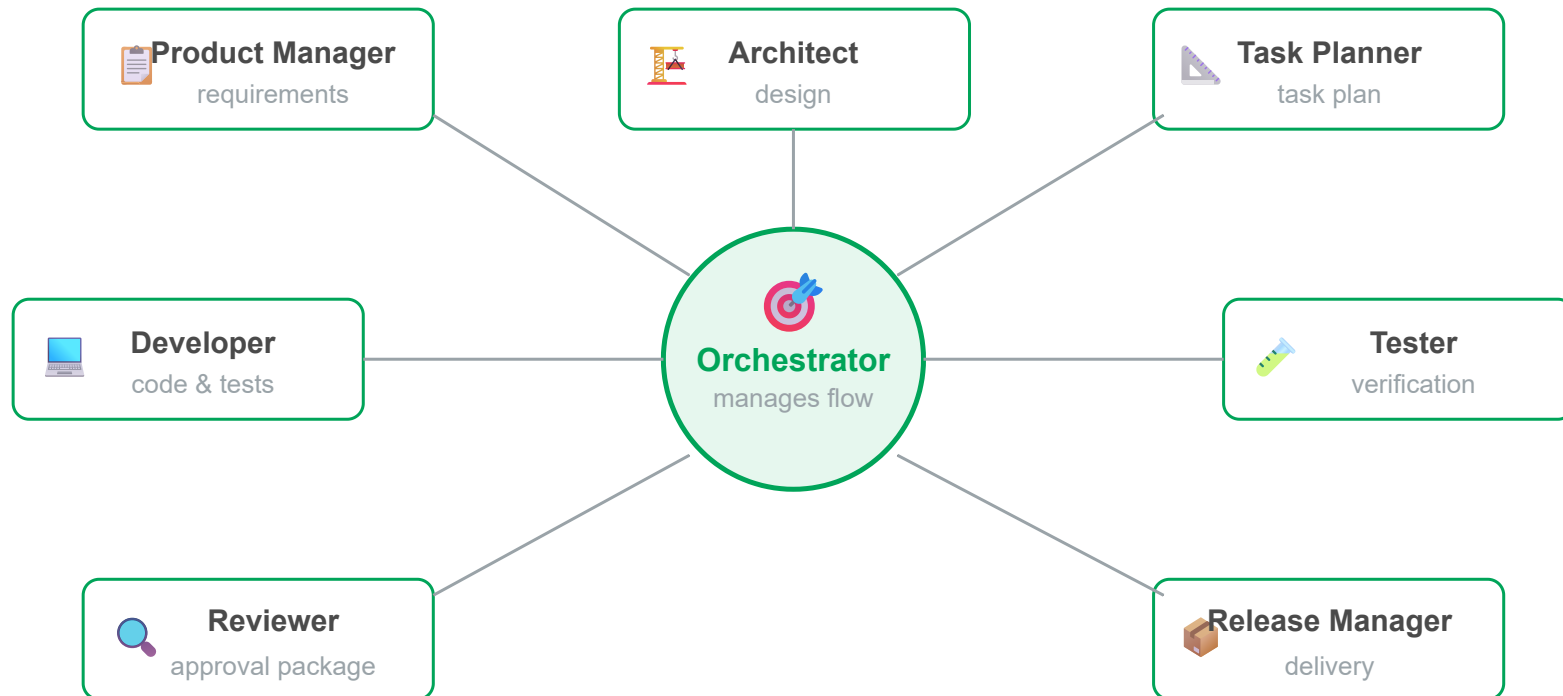
What Happens in Each Phase

Phase	What happens	Key output
 Requirements	User needs are captured and organized into feature groups with clear acceptance criteria	Product Requirements Document
 Architecture	Technical design — components, connections, and risks	Architecture Document
 Planning	One feature group is broken into concrete tasks with dependencies	Task Plan
 Build	Code is written along with automated tests	Source code and test files
 Testing	Tests verify the code meets every acceptance criterion	Test Report
 Review	Summary presented for human approval	Review Package
 Release	Approved increment is versioned and packaged	Release Notes and archive

 If testing finds a problem, the workflow returns to Build for correction — up to **3 times** before a human is asked to step in.

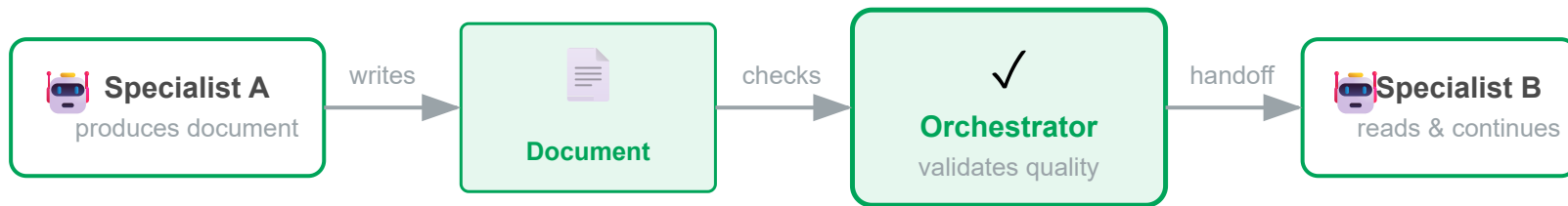
The Team of Specialists

Eight AI specialists participate in the workflow. Each one has a clearly defined responsibility and produces a specific output.



How Specialists Communicate

Specialists do **not** hand work to each other informally. They communicate through documents, and the Orchestrator checks those handoffs before work continues.



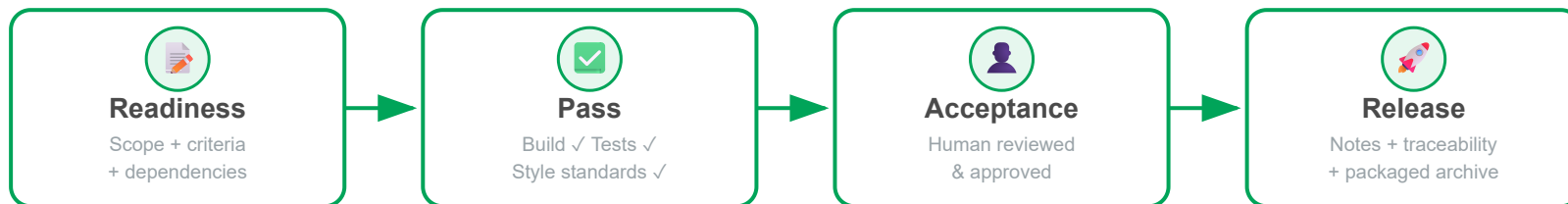
Every decision, requirement, and test result is recorded — nothing is lost.

Each specialist can read earlier documents but can only modify its own.

This document-driven approach creates a **clear audit trail** — every handoff is visible, checked, and recorded.

Quality Gates

These are non-negotiable checkpoints that stop incomplete or low-quality work from moving to the next phase.



If any gate fails, work returns for correction — it never proceeds with known deficiencies.

Requirements Management

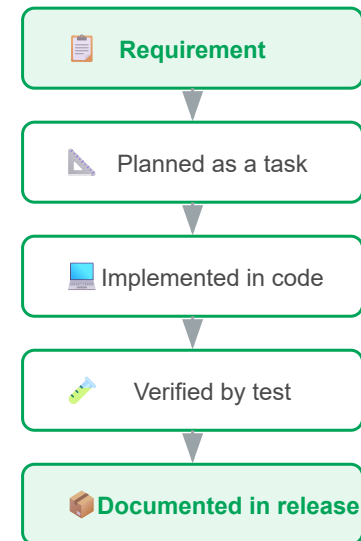
Requirements are the foundation of the workflow. Every planned task, test case, and code change must trace back to a specific requirement.

Two-level structure

- **Feature Groups** — high-level capabilities
e.g. "Card-based gameplay", "Score system"
- **Acceptance Criteria** — specific, testable outcomes
e.g. "Matched pairs stay revealed on the board"

Protective rules

- Identifiers are **assigned once, never reused**
- Retired items are **marked, never deleted**
- Every criterion must have a task, a test, and a code change



Two Operating Modes

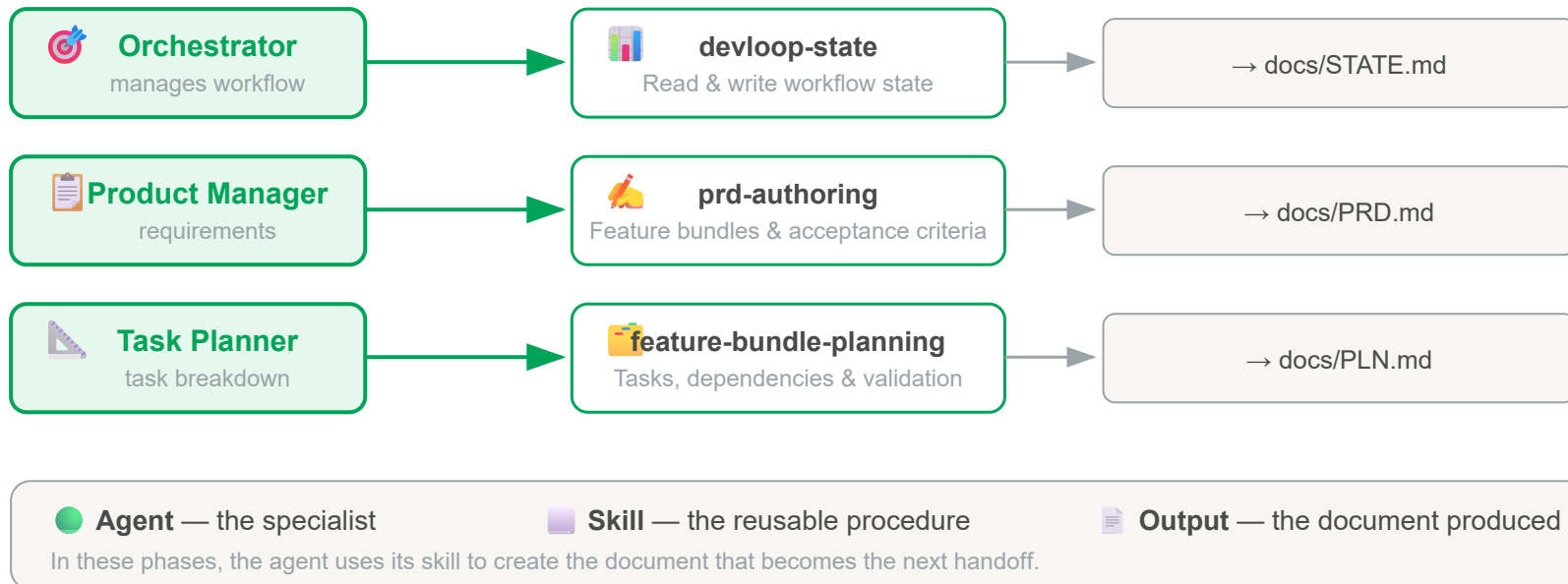
Aspect	 Normal Mode	 Fully Automated Mode
Requirements review	 Human confirms scope & priorities	 Auto-accepted
Architecture review	 Human reviews design	 Auto-accepted
Feature selection	 Human chooses what to build next	 Automatic — next in queue
Review approval	 Human reviews each increment	 Auto-approved
Release	 Optional — human decides when	 Mandatory after every feature
Version numbering	 Human chooses version	 Automatically incremented
Automated quality checks	 Always enforced	 Always enforced

 **Key point:** Even in fully automated mode, quality checks are **always enforced**. Automation removes approval steps, but it does not remove verification.



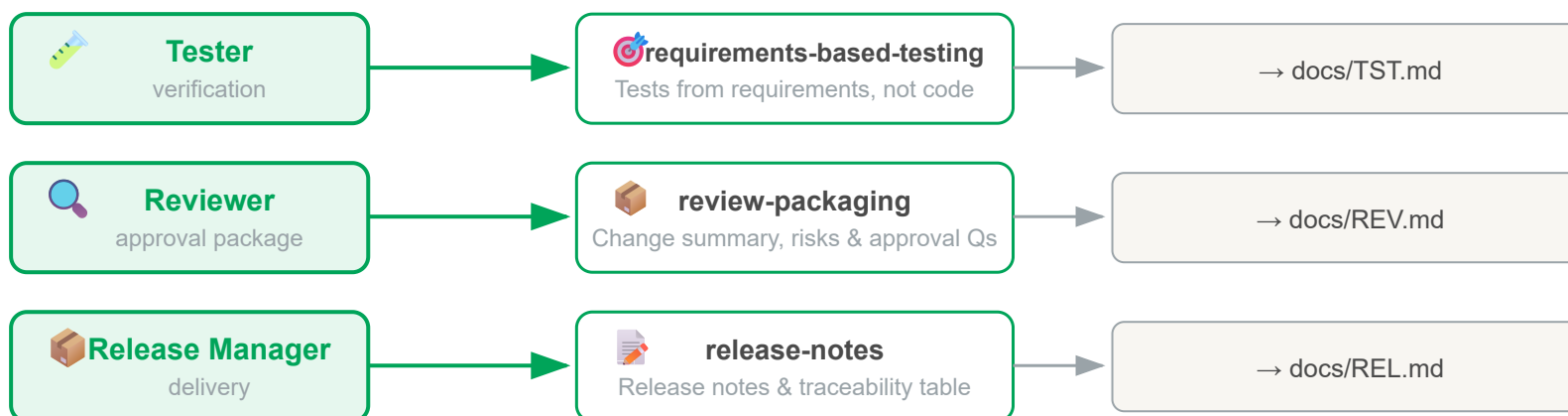
Agent Skills — Agent-to-Skill Mapping (1/2)

Skills are **reusable procedures** that agents invoke to do their job consistently. Each skill packages the instructions needed for one specific task.



Agent Skills — Agent-to-Skill Mapping (2/2)

These are the remaining specialist-to-skill mappings, plus two agents whose procedures are **self-contained** and do not rely on a separate skill file.



⚡ Self-contained agents (no external skill):

 **Architect** — designs modules & interfaces → `docs/ARC.md`  **Developer** — writes code & unit tests → `src/` & `tests/`

Summary



Small Increments

One feature group built,
tested & delivered at a time



Quality by Design

Four quality gates prevent
defects from moving forward



Full Traceability

Every change traces to a
requirement and a test



Clear Accountability

Each specialist owns exactly
one document and one phase



Human Oversight

Approval at key decisions
(unless fully automated)



Documented Decisions

Every phase produces a
persistent document



Build small ·



Verify always ·



Trace everything